

深度学习实践第三次实验报告

李毅PB22051031

1. 思考训练集有多少张图片，测试集有多少张图片。

直接打印信息：

```
# 加载训练集和测试集数据
dataset1 = datasets.MNIST('./data', train=True, download=False,
                           transform=transform)
dataset2 = datasets.MNIST('./data', train=False,
                           transform=transform)

print("训练集样本数量:", len(dataset1))
print("测试集样本数量:", len(dataset2))
```

结果如下：

```
训练集样本数量: 60000
测试集样本数量: 10000
```

2. 修改代码，使用GPU来训练代码，观察前后训练速度变化, 观察跑起来前后显存的变化 (nvidia-smi)

测试环境：CPU intel ultra 7 255HX; GPU Nvidia RTX 5070ti laptop

import time模块用于计时

```
start_time = time.time() # 新增：记录epoch开始时间
train(model, device, train_loader, optimizer, epoch, loss_fn=loss_fn)
acc = test(model, device, test_loader, loss_fn=loss_fn)
epoch_duration = time.time() - start_time # 新增：计算当前epoch运行时间
```

使用CPU速度：

```
Epoch 1 took 10.68 seconds
Epoch 2 took 10.48 seconds
Epoch 3 took 10.39 seconds
Epoch 4 took 10.73 seconds
Epoch 5 took 10.63 seconds
```

使用GPU速度：

```
Epoch 1 took 7.59 seconds
Epoch 2 took 7.67 seconds
Epoch 3 took 7.98 seconds
Epoch 6 took 9.03 seconds
Epoch 7 took 8.01 seconds
```

GPU不比CPU快很多，使用nvidia-smi查看显存变化：

前：

```

+-----+
-----+
| NVIDIA-SMI 572.76                Driver Version: 572.76          CUDA Version:
12.8          |
|-----+-----+-----+
-----+
| GPU  Name                Driver-Model | Bus-Id                Disp.A | volatile
Uncorr. ECC |
| Fan  Temp    Perf          Pwr:Usage/Cap |          Memory-Usage | GPU-Util
Compute M. |
|
| MIG M. |
|=====+=====+=====|
=====|
|  0  NVIDIA GeForce RTX 5070 ...  WDDM |  00000000:01:00.0 off |
N/A |
| N/A   47C    P8              4W / 120W |  180MiB / 12227MiB |  0%
Default |
|
| N/A |
+-----+-----+-----+
-----+

+-----+
-----+
| Processes:
|
| GPU  GI    CI             PID    Type    Process name
GPU Memory |
|      ID    ID
Usage      |
|=====+=====+=====|
=====|
|  0   N/A   N/A             8428    C+G    D:\Softwares\QQ\QQ.exe
N/A      |
+-----+-----+-----+
-----+

```

后:

```

+-----+
-----+
| NVIDIA-SMI 572.76                Driver Version: 572.76          CUDA Version:
12.8          |
|-----+-----+-----+
-----+
| GPU  Name                Driver-Model | Bus-Id                Disp.A | volatile
Uncorr. ECC |
| Fan  Temp    Perf          Pwr:Usage/Cap |          Memory-Usage | GPU-Util
Compute M. |
|
| MIG M. |
|=====+=====+=====|
=====|

```

	0	NVIDIA GeForce RTX 5070 ...	WDDM		00000000:01:00.0	off	
	N/A						
	N/A	47C	P2		21W / 120W		
					673MiB / 12227MiB		
						7%	
Default							
	N/A						
+-----+-----+-----+							
-----+							
+-----+							
-----+							
	Processes:						
	GPU	GI	CI	PID	Type	Process name	
	GPU Memory						
		ID	ID				
Usage							
	=====						
=====							
	0	N/A	N/A	8428	C+G	D:\Softwares\QQ\QQ.exe	
	N/A						
	0	N/A	N/A	15116	C	...odes_python\.conda\python.exe	
	N/A						
	0	N/A	N/A	31680	C	...odes_python\.conda\python.exe	
	N/A						
+-----+-----+-----+							
-----+							

显存出现占用，同时显示了进程信息。

打开任务管理器占用如图：



加速不明显可能是因为模型较为简单。

3. 调大或调小学习率args.lr, 看看有什么变化

截取前5个Epoch的Average loss和Accuracy：

args.lr=0.2时：

Test set: Average loss: 2.3101, Accuracy: 1135/10000 (11%)
Test set: Average loss: 2.3069, Accuracy: 1032/10000 (10%)
Test set: Average loss: 2.3141, Accuracy: 980/10000 (10%)
Test set: Average loss: 2.3119, Accuracy: 982/10000 (10%)
Test set: Average loss: 2.3060, Accuracy: 1009/10000 (10%)

args.lr=0.05（原有值）时：

Test set: Average loss: 0.1423, Accuracy: 9588/10000 (96%)
Test set: Average loss: 0.1218, Accuracy: 9650/10000 (96%)
Test set: Average loss: 0.1255, Accuracy: 9641/10000 (96%)
Test set: Average loss: 0.1095, Accuracy: 9652/10000 (97%)
Test set: Average loss: 0.0941, Accuracy: 9730/10000 (97%)

args.lr=0.01时:

Test set: Average loss:	0.0837,	Accuracy:	9734/10000	(97%)
Test set: Average loss:	0.0615,	Accuracy:	9814/10000	(98%)
Test set: Average loss:	0.0521,	Accuracy:	9832/10000	(98%)
Test set: Average loss:	0.0640,	Accuracy:	9815/10000	(98%)
Test set: Average loss:	0.0495,	Accuracy:	9853/10000	(99%)

高学习率 (0.2)：准确率始终保持在10%左右，较低，损失值稳定在2.3左右（接近初始值）。过大的学习率导致参数更新幅度过大，无法收敛到最优解，在损失平面上震荡或发散

中等学习率 (0.05)： 准确率稳步提升至97%，损失值从0.18稳定下降到0.09

低学习率 (0.01)： 准确率稳步提升至99% (比默认更高)，损失值下降更平缓 (从0.084下降到0.049)

4. 将训练的batchsize改大一些，看看有什么变化

初始batchsize=256:

```
Train Epoch: 1: 100%|  
██████████ | 235/235 [00:07<00:00, 32.42it/s, Loss=0.447, Processed=22464,  
Total=6e+4]  
Test set: Average loss: 0.1870, Accuracy: 9520/10000 (95%)  
  
Train Epoch: 2: 100%|  
██████████ | 235/235 [00:07<00:00, 33.06it/s, Loss=0.464, Processed=22464,  
Total=6e+4]  
Test set: Average loss: 0.1423, Accuracy: 9588/10000 (96%)  
  
Train Epoch: 3: 100%|  
██████████ | 235/235 [00:07<00:00, 32.74it/s, Loss=0.174, Processed=22464,  
Total=6e+4]  
Test set: Average loss: 0.1218, Accuracy: 9650/10000 (96%)  
  
Train Epoch: 4: 100%|  
██████████ | 235/235 [00:07<00:00, 32.47it/s, Loss=0.258, Processed=22464,  
Total=6e+4]  
Test set: Average loss: 0.1255, Accuracy: 9641/10000 (96%)  
  
Train Epoch: 5: 100%|  
██████████ | 235/235 [00:07<00:00, 32.43it/s, Loss=0.21, Processed=22464,  
Total=6e+4]
```

```
Test set: Average loss: 0.1095, Accuracy: 9652/10000 (97%)
```

其他参数为初始值，将batchsize更改到2048：

```
Train Epoch: 1: 100%|
████████████████████████████████████████████████████████████████████████████████
████████████████████████████████████████████████████████████████████████████████| 30/30 [00:06<00:00, 4.32it/s, Loss=2.3, Processed=17632, Total=6e+4]
Test set: Average loss: 2.3014, Accuracy: 1135/10000 (11%)

Train Epoch: 2: 100%|
████████████████████████████████████████████████████████████████████████████████
████████████████████████████████████████████████████████████████████████████████| 30/30 [00:07<00:00, 4.25it/s, Loss=2.3, Processed=17632, Total=6e+4]
Test set: Average loss: 2.3013, Accuracy: 1135/10000 (11%)

Train Epoch: 3: 100%|
████████████████████████████████████████████████████████████████████████████████
████████████████████████████████████████████████████████████████████████████████| 30/30 [00:06<00:00, 4.31it/s, Loss=2.3, Processed=17632, Total=6e+4]
Test set: Average loss: 2.3014, Accuracy: 1135/10000 (11%)

Train Epoch: 4: 100%|
████████████████████████████████████████████████████████████████████████████████
████████████████████████████████████████████████████████████████████████████████| 30/30 [00:07<00:00, 4.27it/s, Loss=2.3, Processed=17632, Total=6e+4]
Test set: Average loss: 2.3017, Accuracy: 1028/10000 (10%)

Train Epoch: 5: 100%|
████████████████████████████████████████████████████████████████████████████████
████████████████████████████████████████████████████████████████████████████████| 30/30 [00:07<00:00, 4.24it/s, Loss=2.3, Processed=17632, Total=6e+4]
Test set: Average loss: 2.3015, Accuracy: 1135/10000 (11%)
```

大批量使得准确率在10%附近稳定，未学习。单epoch训练时间减少。过大的批量导致每个epoch参数更新次数过少（仅30次 vs 原235次）。

5.训练完模型后，观察model.state_dict()，看看里面都有什么

```
import torch
# 加载单个张量的pt文件
loaded_tensor = torch.load('mnist_cnn.pt')
print(loaded_tensor)
```

加载文件，输出如下：

```
OrderedDict([('conv1.weight', tensor([[[[ 0.3332, 0.0958, 0.1755],
      [ 0.4466, -0.0646, -0.0790],
      [ 0.1707, 0.3554, -0.2032]]],
      [[ 0.2456, 0.3074, 0.0843],
      [ 0.4044, 0.1651, 0.2909],
      [ 0.1988, 0.2969, 0.3709]]]]))])
```

```
[[[ 0.0842, -0.0961, -0.2970],
  [-0.4040, -0.2737, -0.3655],
  [-0.3306, -0.2230, -0.0433]]], device='cuda:0')), ('conv1.bias',
tensor([-0.0532,  0.0132, -0.1197], device='cuda:0')), ('conv2.weight',
tensor([[[[-0.2781, -0.1552, -0.2036],
  [-0.5270, -0.5258, -0.3179],
  [-0.1848, -0.3688, -0.2481]],

  [[-0.4174, -0.1868,  0.0208],
  [-0.1836, -0.3309, -0.0985],
  [-0.2292, -0.4304, -0.0924]],

  [[-0.2818, -0.2484, -0.4015],
  [-0.4241, -0.2268, -0.3933],
  [-0.1413, -0.1794, -0.3475]]],

  [[[-0.4576, -0.1099, -0.2694],
  [-0.4535, -0.1027, -0.2154],
  [-0.1583, -0.4361, -0.3636]],

  [[-0.2319, -0.3003, -0.3738],
  [-0.4673, -0.2288, -0.3734],
  [-0.1350, -0.2177, -0.2649]],

  [[-0.2872, -0.2589, -0.2333],
  [-0.5064, -0.2314, -0.4635],
  [-0.3719, -0.2153, -0.2603]]],

  [[[-0.2151, -0.4396, -0.3992],
  [-0.1881, -0.3567, -0.1200],
  [-0.2664, -0.4725, -0.1894]],

  [[-0.2832, -0.3578, -0.4194],
  [-0.2616, -0.2941, -0.4101],
  [-0.3859, -0.1519, -0.2657]],

  [[-0.2202, -0.1454, -0.3526],
  [-0.4921, -0.1440, -0.3183],
  [-0.1966, -0.2885, -0.1665]]],

  [[[-0.0906, -0.1539, -0.4634],
  [-0.2133, -0.0969, -0.0973],
  [-0.1216, -0.3754, -0.3694]],

  [[-0.3456, -0.3147, -0.1738],
  [-0.3475, -0.2447, -0.4892],
  [-0.4219, -0.4162, -0.1969]],

  [[-0.1566, -0.4569, -0.1970],
  [-0.0745, -0.1122, -0.2152],
  [-0.2009, -0.2103, -0.2790]]],
```

```

[[[-0.3632, -0.3905, -0.4180],
  [-0.2566, -0.3547, -0.2636],
  [-0.1823, -0.2905, -0.2685]],

 [[-0.3757, -0.2121, -0.2899],
  [-0.4102, -0.1449, -0.3488],
  [-0.4226, -0.5787, -0.4552]],

 [[-0.4580, -0.4338, -0.1183],
  [-0.4234, -0.5090, -0.1920],
  [-0.1997, -0.2391, -0.4584]]],

 [[[-0.2822, -0.3994, -0.3715],
  [-0.4814, -0.4190, -0.2879],
  [-0.2508, -0.2007, -0.1842]],

 [[-0.2476, -0.1999, -0.2300],
  [-0.3348, -0.4368, -0.2251],
  [-0.4788, -0.3950, -0.4669]],

 [[-0.3951, -0.4681, -0.0590],
  [-0.5285, -0.4015, -0.2477],
  [-0.4615, -0.2751, -0.3091]]], device='cuda:0')), ('conv2.bias',
tensor([-0.4250, -0.3185, -0.1890, -0.4705, -0.2652, -0.2101], device='cuda:0')),
('fc1.weight', tensor([[ 0.0333, -0.3224, -0.2919, ..., -0.2742, -0.2870,
-0.2811],
  [-0.0338, -0.2974, -0.2732, ..., 0.3207, 0.3057, 0.2953],
  [-0.0290, 0.3154, 0.2071, ..., -0.2285, -0.1502, -0.0793],
  ...,
  [ 0.0035, 0.1332, 0.3248, ..., -0.1991, -0.1858, 0.3899],
  [-0.0300, -0.3000, -0.2884, ..., 0.3701, 0.3722, 0.3542],
  [ 0.0219, 0.0339, -0.2920, ..., -0.2689, -0.1927, -0.2055]]],
device='cuda:0')), ('fc1.bias', tensor([-0.3027, -0.3350, -0.1949,
-0.1411, -0.2172, -0.1945, -0.3159, -0.3224,
-0.2851, -0.2481, -0.2371, -0.3305, -0.3129, -0.4716, -0.1648, -0.1177,
-0.1911, -0.2699, -0.3158, -0.3335, -0.1977, -0.2376, -0.2128, -0.1097,
-0.2322, -0.2567, -0.3339, -0.2468, -0.1871, -0.4081, -0.2883, -0.2677,
-0.2242, -0.2184, -0.3295, -0.2916, -0.2343, -0.2984, -0.3166, -0.1949,
-0.3133, -0.2006, -0.2613, -0.3802, -0.3387, -0.1274, -0.3296, -0.1365,
-0.2322, -0.2691, -0.3246, -0.2135, -0.2448, -0.1760, -0.3663, -0.2268,
-0.2296, -0.3131, -0.2493, -0.2203, -0.0857, -0.3386, -0.2118, -0.2744,
-0.1887, -0.3041, -0.3074, -0.3100, -0.2332, -0.1610, -0.2041, -0.1589,
-0.1347, -0.3552, -0.3441, -0.2294, -0.2404, -0.2961, -0.0009, -0.2924,
-0.1748, -0.0536, -0.1156, -0.3521, -0.0947, -0.3150, -0.2834, -0.3162,
-0.4025, -0.2041, -0.2607, -0.3294, -0.3941, -0.3007, -0.3015, -0.4051,
-0.2847, -0.2956, -0.3398, -0.1798, -0.2272, -0.0680, -0.3303, -0.2124,
-0.2593, -0.1123, -0.2766, -0.3032, -0.2633, -0.1697, -0.2144, -0.4090,
-0.3281, -0.3099, -0.0879, -0.2093, -0.1143, -0.1923, -0.2891, -0.3256,
-0.3129, -0.3331, -0.2090, -0.2897, -0.3643, -0.3177, -0.2812, -0.2264],
device='cuda:0')), ('fc2.weight', tensor([[ 0.2267, -0.2554, -0.1750,
..., -0.2880, -0.3051, -0.2063],
  [-0.2612, 0.2367, 0.1669, ..., 0.1455, 0.2361, 0.1367],
  [ 0.2570, 0.3907, 0.2729, ..., 0.2428, 0.3466, 0.1727],

```

```

...,
[ 0.3707, 0.2181, 0.2213, ..., 0.2286, 0.1544, 0.2315],
[ 0.2768, 0.3858, 0.1746, ..., 0.2157, 0.1683, 0.3944],
[ 0.3670, -0.1740, 0.3276, ..., 0.3424, 0.3577, 0.3043]],
device='cuda:0')), ('fc2.bias', tensor([-2.9060e-02, 1.4424e-01,
2.5922e-02, -1.1585e-04, -5.7527e-02,
-9.6713e-02, -1.0953e-02, -1.3277e-02, -6.8680e-02, 5.7288e-03],
device='cuda:0'))))

```

保存了模型的参数数据。

6. 换一个随机数种子，对比一下效果，思考为什么要固定随机数种子

随机数seed更换为10290：输出结果如下：

```

Train Epoch: 1: 100%|
████████████████████████████████████████████████████████████████████████████████
████████████████████████████████████████████████████████████████████████████████ | 235/235
Test set: Average loss: 0.1870, Accuracy: 9520/10000 (95%)

Train Epoch: 2: 100%|
████████████████████████████████████████████████████████████████████████████████
████████████████████████████████████████████████████████████████████████████████ | 235/235 [00:07<00:00, 32.69it/s, Loss=0.464, Processed=22464,
Total=6e+4]
Test set: Average loss: 0.1423, Accuracy: 9588/10000 (96%)

Train Epoch: 3: 100%|
████████████████████████████████████████████████████████████████████████████████
████████████████████████████████████████████████████████████████████████████████ | 235/235 [00:07<00:00, 33.18it/s, Loss=0.174, Processed=22464,
Total=6e+4]
Test set: Average loss: 0.1218, Accuracy: 9650/10000 (96%)

Train Epoch: 4: 100%|
████████████████████████████████████████████████████████████████████████████████
████████████████████████████████████████████████████████████████████████████████ | 235/235 [00:07<00:00, 32.88it/s, Loss=0.258, Processed=22464,
Total=6e+4]
Test set: Average loss: 0.1255, Accuracy: 9641/10000 (96%)

Train Epoch: 5: 100%|
████████████████████████████████████████████████████████████████████████████████
████████████████████████████████████████████████████████████████████████████████ | 235/235 [00:06<00:00, 33.64it/s, Loss=0.21, Processed=22464,
Total=6e+4]
Test set: Average loss: 0.1095, Accuracy: 9652/10000 (97%)

```

可以发现Average loss, Accuracy等属性发生小幅变化。

模型训练中存在很多随机值影响网络的训练结果。固定随机数种子确保网络理论上每一次独立训练的结果都是一样的。

7.用tqdm显示进度条

```
# 训练函数：对单个epoch进行训练
def train(model, device, train_loader, optimizer, epoch, loss_fn):
    model.train() # 将模型设置为训练模式
    pbar = tqdm.tqdm(train_loader, desc=f"Train Epoch: {epoch}")
    for batch_idx, (data, target) in enumerate(pbar):
        data, target = data.to(device), target.to(device) # 将数据移动至计算设备(CPU或GPU)
        optimizer.zero_grad() # 清空梯度
        output = model(data) # 模型前向传播
        loss = loss_fn(output, target) # 计算损失
        loss.backward() # 反向传播计算梯度
        optimizer.step() # 更新模型参数
        pbar.set_postfix({'Loss': loss.detach().item(), 'Processed': batch_idx *
len(data), 'Total': len(train_loader.dataset)}) # 更新进度条信息（用tqdm显示进度条）
```

使用以上代码实现用tqdm显示进度条。本报告中所有进度条都是使用tqdm显示的。

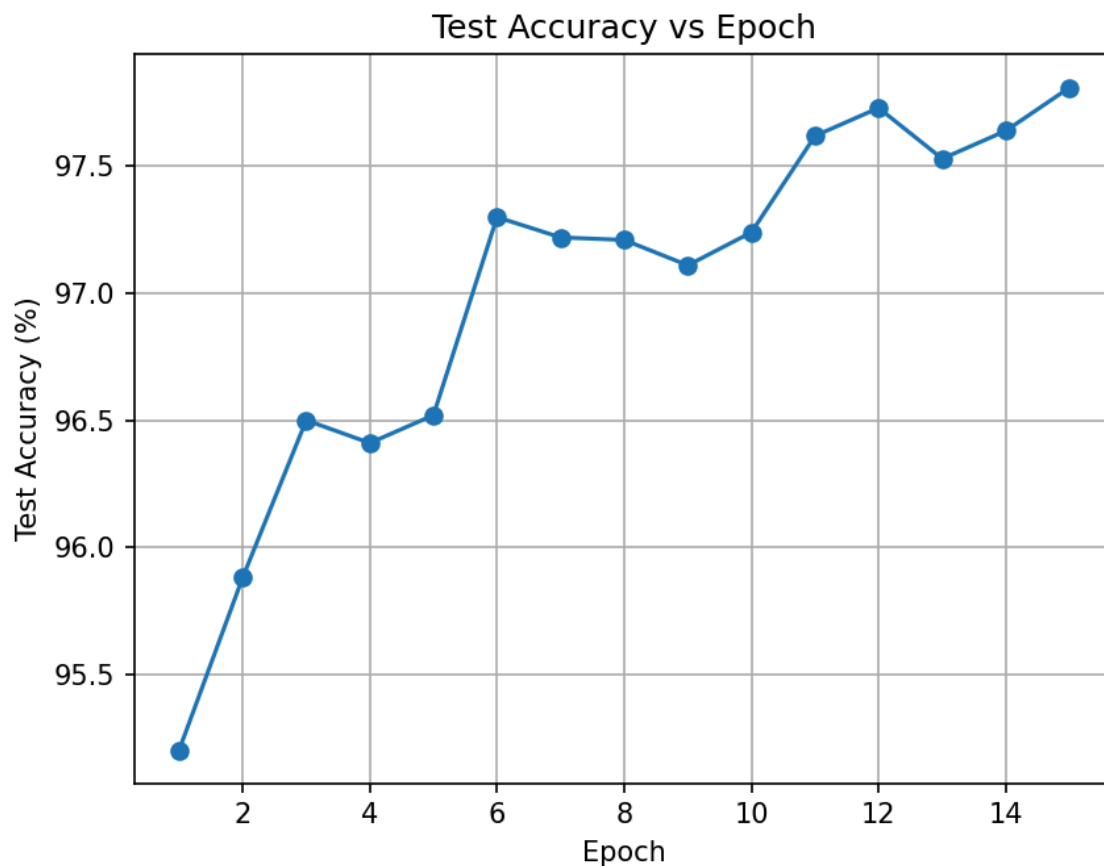
8.修改代码，修改模型Net, 两层卷积后的通道分别为6和12，训练这个模型 (tips: 这时如何调整FC层)

```
def __init__(self):
    super(Net, self).__init__()
    self.conv1 = nn.Conv2d(1, 6, 3, 1)
    self.conv2 = nn.Conv2d(6, 12, 3, 1)
    self.dropout1 = nn.Dropout(0.25)
    self.dropout2 = nn.Dropout(0.5)
    self.fc1 = nn.Linear(1728, 128)
    self.fc2 = nn.Linear(128, 10)
```

如下进行修改。input_size=(1, 1, 28, 28)原始尺寸为28*28，第一层卷积后为26*26。第二层卷积后为24*24。池化后为12*12。现在有12个通道，故fc1输入特征数为1728。

9.修改代码，画图，横坐标是epoch，纵坐标是test集精度结果

记录数据绘图如下：



核心代码:

```
# 绘制测试精度折线图
plt.figure()
plt.plot(epochs_list, test_acc_list, marker='o') # 定义用于记录每个epoch测试精度的列表
plt.xlabel("Epoch")
plt.ylabel("Test Accuracy (%)")
plt.title("Test Accuracy vs Epoch")
plt.grid(True)
plt.show()
```

10.写一个函数，记录下模型测试错误的那些样本并显示出来，看看这些样本都模型推理成了什么

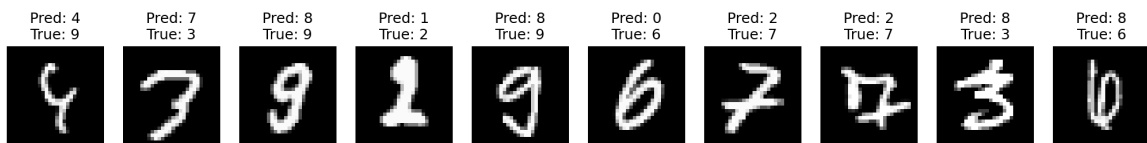
```
def show_misclassified(model, device, test_loader, num_images=10):
    model.eval()
    misclassified = []
    with torch.no_grad():
        for data, target in test_loader:
            data, target = data.to(device), target.to(device)
            output = model(data)
            preds = output.argmax(dim=1)
            # 找出预测错误的样本
            for img, pred, true in zip(data, preds, target):
                if pred.item() != true.item():
                    misclassified.append((img.cpu(), pred.item(), true.item()))
                    if len(misclassified) >= num_images:
                        break
```

```

        if len(misclassified) >= num_images:
            break

# 绘制错误分类的样本
num = len(misclassified)
fig, axes = plt.subplots(1, num, figsize=(num * 3, 3))
if num == 1:
    axes = [axes]
for ax, (img, pred, true) in zip(axes, misclassified):
    img = img.squeeze() # 移除多余的通道维度
    ax.imshow(img, cmap='gray')
    ax.set_title(f"Pred: {pred}\nTrue: {true}")
    ax.axis('off')
plt.show()

```



可以直观看出，这些都是误导性很强的样本。

附录：源代码

```

from __future__ import print_function
from torchvision.datasets import MNIST
from matplotlib import pyplot as plt
import torch # 导入PyTorch核心包
import torch.nn as nn # 导入神经网络模块
import torch.nn.functional as F # 导入常用函数模块，如激活函数、池化函数等
import torch.optim as optim # 导入优化器模块
from torchvision import datasets, transforms # 导入图像数据预处理模块
from torch.optim.lr_scheduler import StepLR # 导入学习率调度器
import numpy as np
import random
import os
import tqdm
import time # 新增：导入time模块用于计时

# 定义卷积神经网络模型 Net，包含两个卷积层、两个Dropout层和两个全连接层
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        # 第一层卷积：输入通道1，输出通道3，卷积核大小3，步长1
        self.conv1 = nn.Conv2d(1, 3, 3, 1)
        # 第二层卷积：输入通道3，输出通道6，卷积核大小3，步长1
        self.conv2 = nn.Conv2d(3, 6, 3, 1)
        # 第一个Dropout层，训练过程中以25%的概率丢弃部分节点
        self.dropout1 = nn.Dropout(0.25)
        # 第二个Dropout层，训练过程中以50%的概率丢弃部分节点
        self.dropout2 = nn.Dropout(0.5)
        # 第一个全连接层：输入特征数864，输出特征数128
        self.fc1 = nn.Linear(864, 128)
        # 第二个全连接层：输入特征数128，输出特征数10（对应10个类别）
        self.fc2 = nn.Linear(128, 10)

```

```

'''
def __init__(self):
    super(Net, self).__init__()
    self.conv1 = nn.Conv2d(1, 6, 3, 1)
    self.conv2 = nn.Conv2d(6, 12, 3, 1)
    self.dropout1 = nn.Dropout(0.25)
    self.dropout2 = nn.Dropout(0.5)
    self.fc1 = nn.Linear(1728, 128)
    self.fc2 = nn.Linear(128, 10)
'''

def forward(self, x):
    # 第一层卷积后接ReLU激活函数
    x = self.conv1(x)
    x = F.relu(x)
    # 第二层卷积后接ReLU激活函数
    x = self.conv2(x)
    x = F.relu(x)
    # 池化层: 使用最大池化操作, 窗口大小为2
    x = F.max_pool2d(x, 2)
    # 第一个Dropout层
    x = self.dropout1(x)
    # 将多维张量扁平化, 保留batch维度
    x = torch.flatten(x, 1)
    # 全连接层1及ReLU激活
    x = self.fc1(x)
    x = F.relu(x)
    # 第二个Dropout层
    x = self.dropout2(x)
    # 全连接层2, 输出类别得分
    x = self.fc2(x)
    return x

# 使用 torchinfo.summary 打印模型结构信息
from torchinfo import summary
print("Model Summary:")
print("-----")
summary(model=Net(), input_size=(1, 1, 28, 28), col_width=20,
        col_names=['input_size', 'output_size', 'num_params', 'trainable'],
        row_settings=['var_names'], verbose=0)
print("-----")

# 设置全局随机数种子, 保证结果可复现
def seed_torch(seed=1029):
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)      # 禁止hash随机化
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)             # 对所有GPU设置随机数种子 (多GPU时)
    torch.backends.cudnn.benchmark = False
    torch.backends.cudnn.deterministic = True

# 训练函数: 对单个epoch进行训练
def train(model, device, train_loader, optimizer, epoch, loss_fn):
    model.train()  # 将模型设置为训练模式

```

```

pbar = tqdm.tqdm(train_loader, desc=f"Train Epoch: {epoch}")
for batch_idx, (data, target) in enumerate(pbar):
    data, target = data.to(device), target.to(device) # 将数据移动至计算设备(CPU或GPU)

    optimizer.zero_grad() # 清空梯度
    output = model(data) # 模型前向传播
    loss = loss_fn(output, target) # 计算损失
    loss.backward() # 反向传播计算梯度
    optimizer.step() # 更新模型参数
    pbar.set_postfix({'Loss': loss.detach().item(), 'Processed': batch_idx *
len(data), 'Total': len(train_loader.dataset)}) # 更新进度条信息 (用tqdm显示进度条)

# 测试函数: 评估模型在测试集上的表现
def test(model, device, test_loader, loss_fn):
    model.eval() # 将模型设置为评估模式
    test_loss = 0
    correct = 0
    with torch.no_grad():
        for data, target in test_loader:
            data, target = data.to(device), target.to(device)
            output = model(data) # 前向传播得到输出
            # 计算整个batch的损失之和 (注意reduction设置为'sum')
            test_loss += loss_fn(output, target, reduction='sum')
            # 得到预测结果: 取每个样本得分最大的类别
            pred = output.argmax(dim=1, keepdim=True)
            # 累加预测正确的样本个数
            correct += pred.eq(target.view_as(pred)).sum().item()

    # 平均损失: 除以测试集样本总数
    test_loss /= len(test_loader.dataset)

    print(test_loss)
    acc = 100. * correct / len(test_loader.dataset)
    print('\nTest set: Average loss: {:.4f}, Accuracy: {}/{} ({:.0f}%) \n'.format(
        test_loss, correct, len(test_loader.dataset),
        acc))
    # 返回测试精度用于绘图
    return acc

# 显示错误分类的样本
def show_misclassified(model, device, test_loader, num_images=10):
    model.eval()
    misclassified = []
    with torch.no_grad():
        for data, target in test_loader:
            data, target = data.to(device), target.to(device)
            output = model(data)
            preds = output.argmax(dim=1)
            # 找出预测错误的样本
            for img, pred, true in zip(data, preds, target):
                if pred.item() != true.item():
                    misclassified.append((img.cpu(), pred.item(), true.item()))
                    if len(misclassified) >= num_images:
                        break
    if len(misclassified) >= num_images:
        break

```

```

num = len(misclassified)
fig, axes = plt.subplots(1, num, figsize=(num * 3, 3))
if num == 1:
    axes = [axes]
for ax, (img, pred, true) in zip(axes, misclassified):
    img = img.squeeze()
    ax.imshow(img, cmap='gray')
    ax.set_title(f"Pred: {pred}\nTrue: {true}")
    ax.axis('off')
plt.show()

# 主函数
def main_ipynb(args, testmodel):
    # 检查是否使用GPU, 如果可用则使用GPU
    use_cuda = not args.no_cuda and torch.cuda.is_available()

    if use_cuda:
        device = torch.device("cuda")
    else:
        device = torch.device("cpu")

    print("device", device)

    # 定义训练和测试的数据加载参数
    train_kwargs = {'batch_size': args.batch_size}
    test_kwargs = {'batch_size': args.test_batch_size}
    if use_cuda:
        # 针对CUDA设置多进程数据加载及pin_memory
        cuda_kwargs = {'num_workers': 1,
                        'pin_memory': True,
                        'shuffle': True}
        train_kwargs.update(cuda_kwargs)
        test_kwargs.update(cuda_kwargs)

    # 预处理
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.1307,), (0.3081,))
    ])

    # 加载训练集和测试集数据
    dataset1 = datasets.MNIST('./data', train=True, download=False,
                              transform=transform)
    dataset2 = datasets.MNIST('./data', train=False,
                              transform=transform)
    print("训练集样本数量:", len(dataset1))
    print("测试集样本数量:", len(dataset2))

    train_loader = torch.utils.data.DataLoader(dataset1, **train_kwargs)
    test_loader = torch.utils.data.DataLoader(dataset2, **test_kwargs)

    # 将模型移动至计算设备
    model = testmodel.to(device)
    # 定义优化器, 这里使用Adam优化器
    optimizer = optim.Adam(model.parameters(), lr=args.lr)

```

```

# 使用交叉熵损失
loss_fn = F.cross_entropy
# 设置学习率衰减：每5个epoch调整一次，衰减因子为args.gamma
scheduler = StepLR(optimizer, step_size=5, gamma=args.gamma)

test_acc_list = []
epochs_list = []

# 开始训练多个epoch
for epoch in range(1, args.epochs + 1):
    start_time = time.time() # 新增：记录epoch开始时间
    train(model, device, train_loader, optimizer, epoch, loss_fn=loss_fn)
    acc = test(model, device, test_loader, loss_fn=loss_fn)
    epoch_duration = time.time() - start_time # 新增：计算当前epoch运行时间
    print("Epoch {} took {:.2f} seconds".format(epoch, epoch_duration)) # 新增：打印epoch运行时长
    test_acc_list.append(acc)
    epochs_list.append(epoch)
    scheduler.step() # 更新学习率

# 绘制测试精度折线图
plt.figure()
plt.plot(epochs_list, test_acc_list, marker='o') # 定义用于记录每个epoch测试精度的列表
plt.xlabel("Epoch")
plt.ylabel("Test Accuracy (%)")
plt.title("Test Accuracy vs Epoch")
plt.grid(True)
plt.show()

# 如果设置了保存模型参数选项，则保存模型到文件
if args.save_model:
    torch.save(model.state_dict(), "mnist_cnn.pt")

show_misclassified(model, device, test_loader, num_images=10) # 显示错误分类的样本

# 设置训练参数类
class MyArgs:
    def __init__(self):
        self.batch_size = 256 # 训练batch大小
        self.test_batch_size = 1000 # 测试batch大小
        self.epochs = 15 # 训练的总epoch数
        self.lr = 0.05 # 初始学习率
        self.gamma = 0.7 # 学习率衰减因子
        self.no_cuda = False # 是否禁用GPU (False表示启用GPU)
        self.seed = 1 # 随机种子，保证实验可复现
        self.log_interval = 10 # 每隔多少个batch输出一次日志
        self.save_model = True # 是否保存训练好的模型

# 程序入口
if __name__ == '__main__':
    args = MyArgs()
    seed_torch(args.seed)

```

```
model = Net()  
main_ipynb(args, model)
```